Implementasi Paralel Estimasi π dengan Metode Monte Carlo

Analisis Komparatif POSIX Threads (Pthreads) vs OpenMP

[Nama] — NIM [NIM]

1. PENDAHULUAN

Latar belakang. Metode Monte Carlo mengestimasi π dengan melempar titik acak (x, y) ke kuadran [0,1]×[0,1] dan menghitung rasio titik yang jatuh di dalam seperempat lingkaran satuan ($x^2+y^2 \le 1$). Karena luas kuadran lingkaran adalah π/4, maka $\pi \approx 4 \cdot hits/N$. Galat estimator ini bersifat stokastik dan menyusut sebagai $O(1/\sqrt{N})$: simpangan baku teoretisnya $\sigma = 4 \cdot \sqrt{(p(1-p)/N)}$ dengan $p = \pi/4$, yaitu ≈ 5.19e-05 untuk $N = 10^9$. Menambah satu digit ketepatan memerlukan 100× lebih banyak sampel, sehingga masalah ini intensif komputasi namun *embarrassingly parallel*: setiap sampel independen dan satu-satunya data bersama hanyalah penjumlahan akhir.

Tujuan. (1) Mengimplementasikan estimasi π dengan $N = 10^9$ sampel dalam tiga versi: serial, Pthreads (manual/eksplisit), dan OpenMP (compiler directive); (2) memastikan correctness yang dapat diverifikasi secara eksak antar versi; (3) mengukur speedup dan efficiency pada 1, 2, 4, dan 8 thread; (4) membandingkan kompleksitas kode, bottleneck, anomali, dan trade-off kedua pendekatan; serta (5) mempelajari dampak kualitas/penyemaian RNG terhadap akurasi statistik.

Lingkungan pengujian. CPU: Intel(R) Xeon(R) Processor @ 2.10GHz; jumlah core logis yang tersedia untuk proses (`nproc`): **1**; thread per core: 1; kompiler: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; flag `-02`; tanggal ukur: 2026-10-31. Setiap konfigurasi dijalankan 5 kali dan dilaporkan **median**-nya (lebih tahan terhadap outlier daripada rerata).

2. DESAIN IMPLEMENTASI

2.1 Algoritma serial (baseline)

Baseline memakai kernel yang *sama persis* dengan versi paralel (file `common.h`), sehingga selisih performa murni berasal dari mekanisme paralelisasi, bukan dari perbedaan algoritma. Kompleksitas waktu O(N), kompleksitas memori O(1) (hanya 32 byte state RNG dan satu penghitung).

```
hits <- 0
untuk b = 0 .. NBLOCKS-1:                # NBLOCKS = 256 blok tetap
    rng <- xoshiro256**(seed = SEED + b*STRIDE)   # stream milik blok b
    untuk i = 1 .. ukuran_blok(b):        # ukuran = N/256 (+1 untuk sisa)
        x <- rng.unit();  y <- rng.unit() # double di [0,1), 53 bit
        hits <- hits + (x*x + y*y <= 1.0) # tanpa branch
pi <- 4 * hits / N
```

Generator acak. Dipakai xoshiro256** (Blackman & Vigna), di-seed lewat SplitMix64. State-nya variabel lokal di stack pemanggil sehingga **tidak ada RNG global yang di-share**. Alternatifnya, `rand_r()` glibc hanya berstate 32 bit dan menghasilkan 31 bit per panggilan sehingga periodenya pendek dan resolusi koordinatnya kasar — tidak ideal untuk 2×10^9 bilangan acak.

Desain reproducibility (keputusan kunci). Jika setiap *thread* diberi stream sendiri, hasil akan berubah mengikuti jumlah thread dan verifikasi terhadap versi serial hanya bisa bersifat statistik. Karena itu sampel dibagi menjadi 256 *blok* tetap, dan **stream RNG dimiliki blok, bukan thread** (seed blok = SEED + b·STRIDE). Thread hanya menentukan siapa yang menghitung blok mana. Konsekuensinya jumlah hit identik bit-per-bit untuk semua versi dan jumlah thread, sehingga verifikasi correctness menjadi perbandingan bilangan bulat yang eksak. Granularitas 256 blok juga cukup halus untuk menyeimbangkan beban pada $T \le 8$ (selisih beban maksimum satu blok = 1/32 dari kerja per thread pada T=8).

2.2 Strategi paralelisasi Pthreads

- **Pembagian workload:** block-cyclic statis. Thread *tid* memproses blok tid, tid+T, tid+2T, ... Tanpa work queue, sehingga tidak ada kontensi saat distribusi kerja.
- **Sinkronisasi: tanpa mutex**. Tiap thread menulis hasil ke slot miliknya (`worker_t.hits`) dan thread utama menjumlahkan setelah `pthread_join`. Join memberi batas sinkronisasi memori menurut POSIX, jadi tidak ada data race. Mutex di loop dalam akan menjadi bottleneck ekstrem (10^9 lock/unlock); mutex di akhir thread pun tidak perlu.
- **Anti false-sharing:** `worker_t` di-pad menjadi 64 byte dan dialokasikan dengan `aligned_alloc(64, …)`, sehingga tiap slot menempati cache line sendiri. Akumulator inti loop adalah variabel lokal (register) dan ditulis ke slot hanya sekali.

```
static void *worker_main(void *p)
{
    worker_t *w = (worker_t *)p;
    uint64_t local = 0;                  /* akumulator di register/stack */
    for (uint64_t b = (uint64_t)w->tid; b < NBLOCKS; b += (uint64_t)w->nthreads) {
        local += mc_block(b, w->n, w->seed);
    }
    w->hits = local;
    return NULL;
}
```

2.3 Strategi paralelisasi OpenMP

```
#pragma omp parallel for schedule(static, 1) reduction(+ : hits)
for (int64_t b = 0; b < (int64_t)NBLOCKS; b++)
    hits += mc_block((uint64_t)b, n, seed);
```

Elemen	Pilihan	Alasan
parallel for	membagi iterasi blok	satu direktif menggantikan create/join/struct/loop manual
schedule(static,1)	blok b → thread (b mod T)	identik dengan Pthreads → perbandingan adil; overhead scheduling nol (beban per blok seragam, sehingga dynamic tidak diperlukan)
reduction(+:hits)	salinan private per thread, digabung di akhir	menghilangkan race condition pada akumulator *dan* false sharing secara otomatis; alternatif atomic/critical akan serialisasi
hits	private (reduction)	diinisialisasi 0 per thread, digabung dengan operator +
b	private (variabel loop)	otomatis private oleh OpenMP
n, seed	shared (read-only)	tidak pernah ditulis → aman tanpa sinkronisasi
state RNG	private (lokal di mc_count)	dideklarasikan di dalam region paralel (stack), bukan variabel global

Pengukuran waktu mencakup seluruh siklus paralel (create/fork → kerja → join/reduksi) pada kedua versi, sehingga overhead manajemen thread ikut terhitung secara adil.

3. HASIL PENGUJIAN

3.1 Verifikasi correctness

Skrip `scripts/test.sh` membandingkan jumlah hit Pthreads dan OpenMP (T = 1, 2, 3, 4, 8) dengan versi serial untuk N = 10^7, 10 000 019 (tidak habis dibagi 256, menguji penanganan sisa), 1000, dan 255. Seluruhnya identik (perbandingan integer, bukan toleransi). Pada benchmark utama:

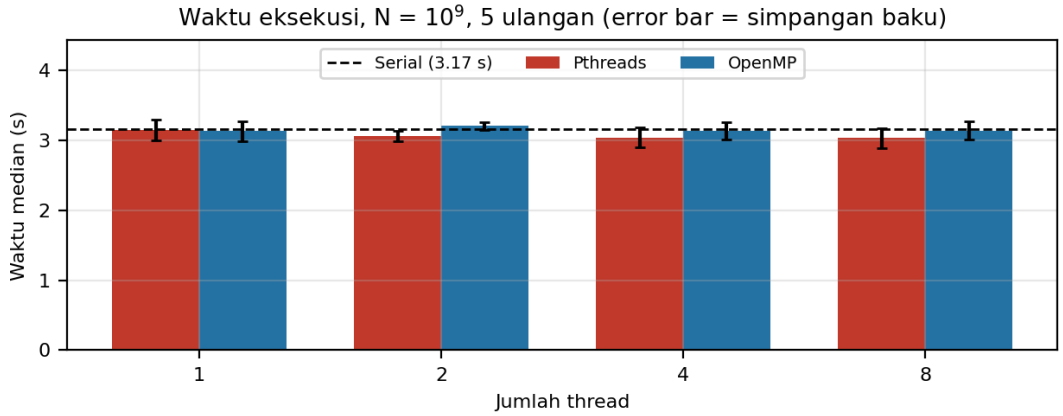
Pemeriksaan	Hasil
Hit identik di semua versi & semua T pada N = 10^9	**LULUS** — semua 45 proses menghasilkan hits = 785402336
Estimasi π	3.1416093440 (|galat| = 1.669e-05)
Simpangan baku teoretis σ	5.193e-05 → galat = 0.32 σ (wajar, < 3σ)

Karena masalah Monte Carlo tidak punya jawaban “10×10 yang bisa dihitung manual”, verifikasi dilakukan dua lapis: (a) kesetaraan eksak dengan versi serial, dan (b) uji statistik bahwa estimasi berada dalam 5σ teoretis. Lapis (a) membuktikan paralelisasi tidak mengubah komputasi; lapis (b) membuktikan komputasi dasarnya benar.

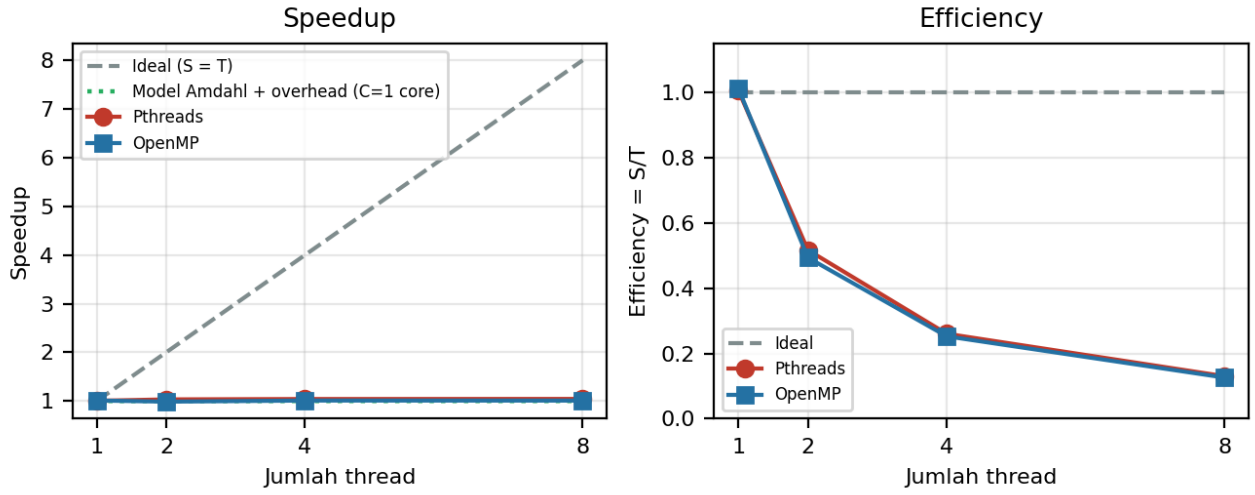
3.2 Waktu eksekusi, speedup, dan efficiency

Implementasi	T	Median (s)	Min (s)	σ rel.	Speedup	Efficiency
Serial	1	3.169	2.985	3.6%	1.00	—
Pthreads	1	3.155	2.896	4.7%	1.00	1.00
Pthreads	2	3.067	3.055	2.3%	1.03	0.52
Pthreads	4	3.044	2.999	4.8%	1.04	0.26
Pthreads	8	3.038	2.977	4.7%	1.04	0.13
OpenMP	1	3.134	3.002	4.6%	1.01	1.01
OpenMP	2	3.211	3.095	1.7%	0.99	0.49
OpenMP	4	3.138	3.001	3.8%	1.01	0.25
OpenMP	8	3.142	2.956	4.1%	1.01	0.13

Tabel 1. Hasil pengukuran (N = 10^9, 5 ulangan). Speedup $S = T_{serial}/T_{paralel}$ (median); Efficiency $E = S/T$.



Gambar 1. Waktu eksekusi median terhadap jumlah thread.



Gambar 2. Speedup (kiri) dan efficiency (kanan). Garis titik hijau: model Amdahl dengan overhead terukur T(t) = overhead(t) + T_{serial}/min(t, C) untuk C = 1 core.

3.3 Overhead pembuatan thread (N = 1000)

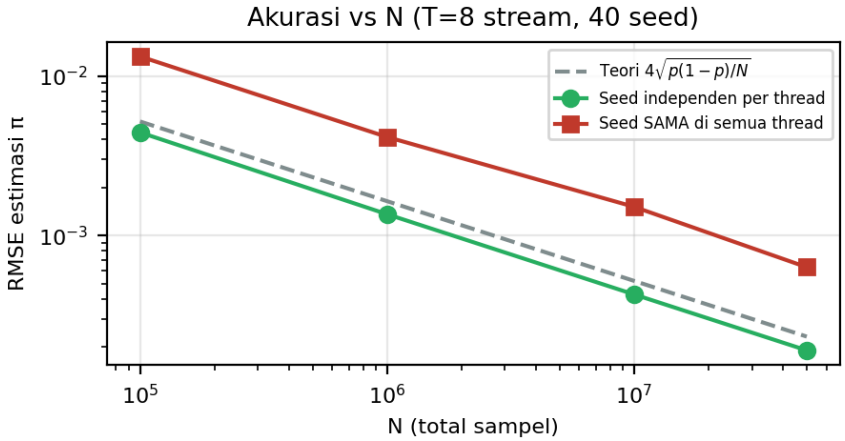
T	1	2	4	8
Pthreads (µs)	66	93	127	193
OpenMP (µs)	8	58	101	182

Tabel 2. Median waktu total saat N = 1000 (kerja sebenarnya ≈ 0), sehingga hampir seluruhnya adalah biaya create/join (Pthreads) atau fork/join + reduksi (OpenMP).

3.4 Akurasi statistik terhadap penyemaian RNG

N	Seed	σ teori	RMSE independen	RMSE seed sama	Rasio
10^5	40	5.19e-03	4.44e-03	1.33e-02	3.00
10^6	40	1.64e-03	1.36e-03	4.15e-03	3.05
10^7	40	5.19e-04	4.27e-04	1.51e-03	3.54
5×10^7	40	2.32e-04	1.90e-04	6.33e-04	3.32

Tabel 3. RMSE estimasi π dari 40 seed berbeda, dengan 8 stream (T = 8).

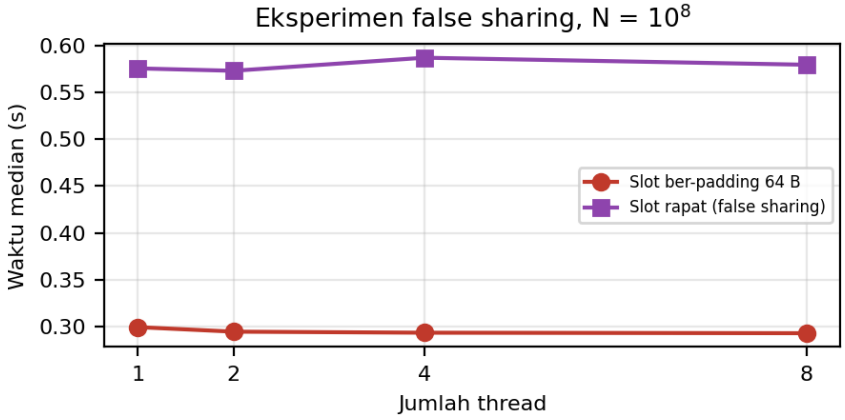


Gambar 3. RMSE mengikuti kurva teoretis 1/√N bila stream independen; bila semua thread memakai seed sama, galat lebih besar.

3.5 Eksperimen false sharing

T	Padded (s)	Rapat/false sharing (s)	Rasio	Rasio ternormalisasi
1	0.300	0.576	1.92	1.00
2	0.295	0.573	1.94	1.01
4	0.294	0.587	2.00	1.04
8	0.293	0.579	1.98	1.03

Tabel 4. Varian `mc_pthreads_fs` menaikkan penghitung di memori pada setiap hit (volatile, slot rapat 8 byte). "Rasio ternormalisasi" = rasio/rasio(T=1) memisahkan efek false sharing dari biaya akses volatile murni.



Gambar 4. Waktu median varian ber-padding vs varian false sharing.

4. ANALISIS KRITIS

4.1 Perbandingan performa Pthreads vs OpenMP

Selisih waktu median Pthreads terhadap OpenMP (positif = Pthreads lebih lambat): T=1: +0.6% (ambang noise ±9.4%); T=2: -4.5% (ambang noise ±4.7%); T=4: -3.0% (ambang noise ±9.5%); T=8: -3.3% (ambang noise ±9.4%). Pada semua T, selisih berada di dalam ±2σ variasi antar-ulangan, sehingga **tidak ada bukti bahwa salah satu pendekatan lebih cepat**. Ini sesuai ekspektasi teoretis: kedua versi menjalankan kernel yang sama dengan pembagian kerja yang sama, dan di balik layar OpenMP pada GCC (libgomp) juga memakai thread POSIX. Perbedaan hanya terletak pada kode pengelolaan thread yang jumlahnya O(T) dan terjadi sekali.

4.2 Kompleksitas kode dan maintenance

Metrik	Serial	Pthreads	OpenMP
LOC efektif file utama (tanpa komentar/kosong, tanpa varian FS)	13	46	16
LOC tambahan vs serial	—	+33	+3
Simbol API/direktif paralel unik	0	3	2
Struct/alokasi manual, padding cache-line	tidak ada	ya (worker_t, aligned_alloc)	tidak ada
Penanganan error	—	manual (rc pthread_create)	dikelola runtime
Kernel bersama (common.h + rng.h)	110	sama	sama

Tabel 5. Metrik kompleksitas (dihitung otomatis oleh scripts/make_report.py dari berkas src/).

Untuk masalah ini, OpenMP hanya menambah **3** baris kode efektif terhadap serial, sedangkan Pthreads menambah **33** baris (≈ 11.0× lebih banyak). Dari sisi *maintenance*: (i) versi OpenMP dapat dibaca sebagai program serial dengan satu anotasi dan dapat dikompilasi tanpa `-fopenmp` menjadi serial yang benar (*incremental parallelism*), (ii) versi Pthreads menyebar ke struct argumen, fungsi worker, loop create dan loop join, sehingga perubahan strategi pembagian kerja menyentuh beberapa tempat, dan (iii) Pthreads menuntut programmer sendiri menjamin tidak ada data race dan false sharing — kesalahan yang tidak terdeteksi kompiler dan sering hanya muncul sebagai performa buruk atau hasil nondeterministik. Konstruk `reduction` membuat seluruh kelas bug itu mustahil terjadi pada versi OpenMP.

4.3 Analisis bottleneck dan hukum Amdahl

Hukum Amdahl: S(T) = 1/(f + (1−f)/T) dengan f fraksi yang tidak dapat diparalelkan. Fraksi serial di sini hanya pembuatan/penggabungan thread dan reduksi akhir. Dari Tabel 2, overhead pada T = 8 sebesar 193 µs (Pthreads) dan 182 µs (OpenMP) terhadap T_{serial} = 3.17 s, yaitu f ≈ 6.1e-05 dan 5.8e-05. Batas Amdahl 1/f ≈ 1.6e+04 (sangat besar), sehingga **secara algoritmik tidak ada bottleneck serial**; speedup teoretis pada T = 8 praktis sama dengan T bila ada ≥ 8 core fisik.

Kernel ini *compute-bound* murni: working set 32 byte muat di register/L1, tidak ada trafik memori berarti, sehingga bandwidth DRAM dan cache locality bukan faktor. Throughput dibatasi rantai dependensi xoshiro (≈ 3.17 ns per sampel pada serial). Karena N tetap dan T bertambah, ini *strong scaling*; untuk *weak scaling* (hukum Gustafson), memperbesar N sebanding T akan menjaga waktu tetap sambil menurunkan galat sebesar √T.

4.4 Pembahasan anomali

Anomali utama: speedup tidak mengikuti T. Speedup maksimum yang teramati hanya 1.04× padahal T hingga 8 (selisih dari 1.00 masih dalam variasi antar-ulangan, bukan percepatan nyata), dan efficiency turun sebagai ≈ 1/T (Tabel 1). Ini **bukan cacat implementasi**: mesin ukur hanya menyediakan **1 core logis**, sehingga T > 1 thread hanya *di-time-slice* pada core yang sama (oversubscription). Total kerja tetap sama, maka waktu tetap ≈ T_{serial} ditambah overhead konteks-switch. Model Amdahl dengan min(T, C) = 1 (garis hijau Gambar 2) memprediksi perilaku ini dengan baik, yang menjadi validasi bahwa penyebabnya keterbatasan hardware, bukan kode. Bukti pendukung: hit identik di semua T (kerja tidak hilang/dobel) dan overhead thread sangat kecil (Tabel 2). **Keterbatasan:** angka speedup > 1 tidak dapat didemonstrasikan pada mesin ini; benchmark harus dijalankan ulang (`make bench`) pada mesin ≥ 8 core, dan laporan ini otomatis ter-regenerasi dengan data tersebut.

Variabilitas. Simpangan baku relatif terbesar adalah 4.8% (pthreads, T=4); variasi ini berasal dari noise lingkungan (host virtual, penjadwal OS) sehingga selisih di bawah nilai ini tidak boleh ditafsirkan sebagai keunggulan implementasi tertentu.

False sharing tidak teramati. Rasio ternormalisasi pada T = 8 adalah 1.03. False sharing membutuhkan beberapa core yang menulis cache line yang sama secara bersamaan; dengan hanya 1 core tidak ada ping-pong cache-coherence. Selisih rasio T=1 hanyalah biaya store ke memori volatile. Pada mesin multi-core, varian ini diperkirakan melambat nyata dan rasio ternormalisasi akan naik jauh di atas 1; itulah alasan slot pada versi utama di-pad ke 64 byte.

4.5 Trade-off jumlah thread vs akurasi statistik

Untuk N total tetap, jumlah thread **tidak** mengubah galat statistik selama stream independen: galat hanya bergantung pada N ($\sigma = 4\sqrt{(p(1-p)/N)}$). Percobaan 3.4 menunjukkan RMSE seed-independen berada di sekitar kurva teoretis (rasio RMSE/σ = 0.82–0.85; ketidakpastian sampling RMSE dari 40 seed ≈ ±11%; mis. $N = 5\times10^7$: RMSE 1.90e-04 vs σ 2.32e-04). Sebaliknya, **bug seed sama** di 8 thread memberi 8 salinan sampel yang identik; informasi efektif hanya N/8, sehingga galat membesar ≈ √8 = 2.83× (terukur 3.00–3.54×, ketidakpastian ≈ ±16%) — hasil tampak "jalan" dan cepat, tetapi tidak lebih akurat daripada satu thread. Ini menegaskan bahwa speedup dari paralelisme hanya bermakna bila independensi statistik dijaga. Memakai seed berurutan (seed, seed+1, …) di `rand_r` juga berisiko korelasi antar stream; desain kami menurunkan seed lewat SplitMix64 yang mengacak bit sebelum mengisi state xoshiro.

Pilihan lain yang layak untuk RNG paralel yang lebih kuat adalah *jump-ahead* xoshiro (fungsi `jump()` yang melompat 2^{128} langkah) agar stream dijamin tidak tumpang tindih; kami memakai penyemaian SplitMix64 karena tumpang tindih pada state 256 bit secara praktis tidak mungkin (256 stream × ≈ 4×10^6 sampel per stream ≪ 2^{256}).

4.6 Trade-off kemudahan vs kontrol

Aspek	Pthreads	OpenMP
Kontrol atas thread (afinitas, prioritas, siklus hidup, pola komunikasi)	Penuh; dapat mengimplementasikan pipeline, work-queue, producer-consumer	Terbatas pada model fork-join & task; sisanya via environment variable/clause
Jumlah kode & risiko bug	Tinggi; race/false sharing/leak tanggung jawab programmer	Rendah; reduction, private, schedule dikelola compiler
Portabilitas	POSIX (Windows butuh lapisan tambahan)	Standar lintas-kompiler/OS; hanya perlu flag
Evolusi kode	Mengubah strategi → ubah banyak tempat	Mengubah schedule/clause → satu baris
Cocok untuk	Konkurensi tak-beraturan, I/O, event loop, kontrol tingkat rendah	Loop numerik paralel data (HPC), prototipe cepat

Dalam masalah ini kontrol ekstra Pthreads **tidak memberi keuntungan terukur** (4.1), sedangkan biayanya adalah kode yang lebih panjang dan lebih rawan error (4.2). Kontrol Pthreads baru berharga pada pola yang tidak dapat diekspresikan sebagai loop paralel, misalnya pipeline producer–consumer atau thread yang hidup lama.

5. KESIMPULAN

Rekomendasi.

- Gunakan **OpenMP** untuk komputasi numerik berbasis loop yang *embarrassingly parallel* seperti Monte Carlo: performa setara, kode jauh lebih singkat, dan `reduction` menghapus race condition.
- Gunakan **Pthreads** bila memerlukan kontrol siklus hidup/komunikasi thread atau pola konkurensi non-loop; siapkan disiplin: akumulator lokal, padding cache-line, dan sinkronisasi hanya di batas join.

Pembelajaran.

- Menjadikan unit kerja (blok) pemilik stream RNG, bukan thread, membuat hasil paralel *reproducible* dan dapat diverifikasi eksak terhadap serial.
- Speedup dibatasi oleh hardware yang tersedia, bukan hanya oleh kode: tanpa memeriksa jumlah core, kurva speedup dapat disalahartikan sebagai bug. Model Amdahl dengan overhead terukur dan min(T, C) menjelaskan data dengan baik.
- Independensi RNG antar thread menentukan apakah paralelisme benar-benar menambah informasi statistik.
- Pengukuran harus berulang dan dilaporkan dengan variabilitasnya agar selisih kecil tidak disalahtafsirkan.

6. DAFTAR PUSTAKA

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, 483–485.

[2] Blackman, D. & Vigna, S. (2021). Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4), 1–32.

[3] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.

[4] Chapman, B., Jost, G. & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[5] Gustafson, J. L. (1988). Reevaluating Amdahl’s law. *Communications of the ACM*, 31(5), 532–533.

[6] Metropolis, N. & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341.

[7] OpenMP Architecture Review Board (2021). *OpenMP Application Programming Interface, Version 5.2*.

[8] Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.

[9] Steele, G. L., Lea, D. & Flood, C. H. (2014). Fast splittable pseudorandom number generators. *ACM SIGPLAN Notices (OOPSLA)*, 49(10), 453–472.